# Coding challenge 6: Iterations and functions

Dulshika Dassanayake

2026-03-23

## Contents

**1 Reproducibility** **1**

**2 Conceptual Explanation** **1**

**3 Read the CSV** **2**

**4 Function writing** **3**

**5 Calculate the distance between Auburn and New York** **3**

**6 for loop to calculate the distance between Auburn, AL and every other city in Cities.csv** **4**

**7 Modify loop** **5**

**8 GitHub** **5**

## 1 Reproducibility

Writing own functions and iterations helps ensure reproducibility by making the analysis systematic and repeatable. Functions apply the same instructions each time, while iterations automate repeating the same process across multiple datasets. This reduces human error and ensures that results can be reproduced exactly.

## 2 Conceptual Explanation

A function in R is written using the function() command. The inputs (arguments) are placed inside the parentheses, and the code that performs the calculation is written inside curly brackets { }. The function processes the inputs and returns the result using return() or by outputting the final value.

A for loop in R is written using the syntax for (i in sequence). The loop runs the same block of code repeatedly for each element in the sequence. The code inside the curly brackets executes during each iteration, and results are typically stored in objects such as vectors or data frames.

# 3 Read the CSV

```r
library(ggplot2)
library(drc)
```

```
## Loading required package: MASS

##
## 'drc' has been loaded.

## Please cite R and 'drc' if used for a publication,

## for references type 'citation()' and 'citation('drc')'.

##
## Attaching package: 'drc'

## The following objects are masked from 'package:stats':
##
##     gaussian, getInitial
```

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v lubridate 1.9.4     v tibble    3.3.1
## v purrr     1.2.1     v tidyr     1.3.2

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## x dplyr::select() masks MASS::select()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
citydata=read.csv ("Cities.csv", na.strings = "na")
head(citydata)
```

```
##          city  city_ascii state_id state_name county_fips county_name     lat
## 1    New York    New York       NY   New York       36081      Queens 40.6943
## 2 Los Angeles Los Angeles       CA California        6037 Los Angeles 34.1141
## 3     Chicago     Chicago       IL   Illinois       17031        Cook 41.8375
## 4       Miami       Miami       FL    Florida       12086  Miami-Dade 25.7840
## 5     Houston     Houston       TX      Texas       48201      Harris 29.7860
## 6      Dallas      Dallas       TX      Texas       48113      Dallas 32.7935
##        long population density
## 1  -73.9249   18832416 10943.7
## 2 -118.4068   11885717  3165.8
## 3  -87.6866    8489066  4590.3
## 4  -80.2101    6113982  4791.1
## 5  -95.3885    6046392  1386.5
## 6  -96.7667    5843632  1477.2
```

# 4 Function writing

```
distance <- function(lat1, lon1, lat2, lon2){
  #convert to radians
  rad.lat1 <- lat1 * pi/180
  rad.lon1 <- lon1 * pi/180
  rad.lat2 <- lat2 * pi/180
  rad.lon2 <- lon2 * pi/180

  #Haversine formula
  delta_lat <- rad.lat2 - rad.lat1
  delta_lon <- rad.lon2 - rad.lon1
  a <- sin(delta_lat / 2)^2 + cos(rad.lat1) * cos(rad.lat2) * sin(delta_lon / 2)^2
  c <- 2 * asin(sqrt(a))

  #Earth's radius in km
  earth_radius <- 6378137

  #Calculate the distance
  distance_km <- (earth_radius * c)/1000
  return(distance_km)
}
```

# 5 Calculate the distance between Auburn and New York

```
Au_NY <- function(start, destination)
{
  cord_extract <- citydata %>%
  select(city, lat, long) %>%
  filter(city == start | city == destination) %>%
  group_by(city)
head(cord_extract)
cords <- list(cord_extract$lat, cord_extract$long)
latitude_1 <- cords[[1]][1]
latitude_2 <- cords[[1]][2]
longitude_1 <- cords[[2]][1]
longitude_2 <- cords[[2]][2]
return(distance(latitude_1, longitude_1, latitude_2, longitude_2))
}

Au_NY ("New York", "Auburn")
```

```
## [1] 1367.854
```

# 6 for loop to calculate the distance between Auburn, AL and every other city in Cities.csv

```
Au_to_cities <- unique(citydata$city)
for (i in seq_along(Au_to_cities)) {
  results <- Au_NY(Au_to_cities[[i]], "Auburn" )
  print(results)
}
```

```
## [1] 1367.854
## [1] 3051.838
## [1] 1045.521
## [1] 916.4138
## [1] 993.0298
## [1] 1056.022
## [1] 1239.973
## [1] 162.5121
## [1] 1036.99
## [1] 1665.699
## [1] 2476.255
## [1] 1108.229
## [1] 3507.959
## [1] 3388.366
## [1] 2951.382
## [1] 1530.2
## [1] 591.1181
## [1] 1363.207
## [1] 1909.79
## [1] 1380.138
## [1] 2961.12
## [1] 2752.814
## [1] 1092.259
## [1] 796.7541
## [1] 3479.538
## [1] 1290.549
## [1] 3301.992
## [1] 1191.666
## [1] 608.2035
## [1] 2504.631
## [1] 3337.278
## [1] 800.1452
## [1] 1001.088
## [1] 732.5906
## [1] 1371.163
## [1] 1091.897
## [1] 1043.273
## [1] 851.3423
## [1] 1382.372
## [1] NA
```

# 7 Modify loop

```
all_distances <- NULL
Au_to_cities <- unique(citydata$city)
for (i in seq_along(Au_to_cities)) {
  results <- Au_NY(Au_to_cities[[i]], "Auburn" )
  distance_i <- data.frame(Au_to_cities[[i]], "Auburn", results)
  colnames(distance_i) <- c("Destination", "Origin", "Distance (Km)")
  all_distances <- rbind.data.frame(all_distances, distance_i)
}

head(all_distances)
```

```
##   Destination Origin Distance (Km)
## 1    New York Auburn     1367.8540
## 2 Los Angeles Auburn     3051.8382
## 3     Chicago Auburn     1045.5213
## 4       Miami Auburn      916.4138
## 5     Houston Auburn      993.0298
## 6      Dallas Auburn     1056.0217
```

# 8 GitHub

My GitHub link: